

Secure Vault: A Hardware-Bound File Encryption System Using USB-Based Key Custody

Keerthivasan S

Independent Researcher

github.com/keerthivasan-sankar/secure-vault-for-commercial

August 2026

Abstract

Password-based file encryption remains vulnerable to guessing, phishing, keylogging, and credential-stuffing attacks that exploit the fact that a secret held only in human memory is both weak and remotely exposed. This paper presents Secure Vault, an open-source, local-first file encryption system for Windows that replaces password-only key custody with a physical USB flash drive as the primary key-bearing token, combined with AES-256-GCM authenticated encryption and scrypt-based key derivation. We describe the system’s threat model, cryptographic design, and implementation, and we present a chronological account of its security hardening process across six releases (v3.0 through v3.5.0), during which a plaintext key-storage flaw, a forgeable unkeyed checksum, a shell command-injection vector, key-reuse across cryptographic purposes, and cross-context ciphertext substitution were identified and remediated. We discuss the trade-offs of USB-based key custody — including the fact that it constitutes removable-key possession rather than hardware-bound secret storage, and the absence of password recovery by design — and report on continuous static analysis via GitHub CodeQL and dependency monitoring via Dependabot as a substitute for a formal third-party audit, which the project has not yet undergone. We conclude that physical-token-based key custody is a practical, low-cost hardening mechanism for individual users and small teams, while identifying code signing, independent audit, and migration toward genuine hardware-bound secret storage as the primary remaining gaps before production deployment at scale.

Keywords: file encryption, AES-256-GCM, scrypt, HKDF, two-factor authentication, USB key, hardware token, Node.js, applied cryptography

1 Introduction

Full-disk and file-level encryption tools such as BitLocker and VeraCrypt typically rely on a password, a PIN, or a TPM-bound key as the sole or primary secret protecting user data. Each of these mechanisms has a well-documented weakness: passwords are guessable or phishable, PINs are short and brute-forceable, and TPM-bound keys tie data irrevocably to a specific machine. Meanwhile, consumers and students who handle sensitive personal files — academic records, financial documents, identification scans — rarely have access to enterprise key-management infrastructure, and often reuse or forget passwords used for encryption.

Secure Vault addresses this gap with a deliberately narrow design goal: encrypt individual files or folders on a single Windows machine, using a randomly generated key that is never derived from, or reducible to, something the user has memorized. Instead, the key is generated at registration time and stored — itself encrypted — on a USB flash drive that the user designates as their vault key. Decryption is only possible when that physical drive is present and the user supplies the password used to unlock the key material stored on it. This paper documents the design, implementation, and iterative security hardening of that system through its current release, v3.5.0.

1.1 Contributions

This paper makes the following contributions:

1. A description of a practical hardware-token-based file encryption design suitable for individual, non-enterprise users, implemented entirely with standard library and minimal third-party dependencies.
2. A documented threat model and explicit statement of the trade-offs introduced by removing password-only recovery, including a precise characterization of USB-based custody as removable-key possession rather than tamper-resistant hardware-bound storage.
3. A chronological security hardening case study spanning six point releases, in which five distinct vulnerability and design-hardening classes — plaintext key exposure, checksum forgery, command injection, cross-purpose key reuse, and cross-context ciphertext substitution — were identified through review and fixed, illustrating a realistic maintenance lifecycle for a solo-developed security tool.
4. A discussion of the role of continuous static analysis (GitHub CodeQL, Dependabot) as an interim assurance mechanism in the absence of a funded third-party audit.

2 Related Work

Commercial and open-source disk/file encryption tools generally fall into three categories. Software-only, password-derived key tools (e.g., VeraCrypt, 7-Zip AES encryption) derive the encryption key entirely from a user-supplied passphrase, typically via PBKDF2 or a similar key-derivation function; security is bounded by passphrase strength and by the user’s ability to keep it secret. TPM-bound tools (e.g., Windows BitLocker in its default configuration) bind the key to a specific machine’s Trusted Platform Module, which protects against offline disk theft but ties recoverability to that specific hardware and depends on firmware/TPM integrity. Hardware security key tools (e.g., YubiKey-based FDE unlock, or commercial products such as IronKey) use a dedicated cryptographic token, often over FIDO2/PIV, which offers strong tamper resistance — raw key material never leaves the device — but requires purchasing specialized hardware and, for many products, vendor lock-in.

Secure Vault occupies a fourth position: it uses commodity USB flash storage — not a dedicated cryptographic token — as a bearer object for a locally generated key, combined with a password that unlocks that key. This is a two-factor design (something you have plus something you know) built entirely from primitives already present on a standard Windows machine and a Node.js runtime, at zero hardware cost beyond a spare USB drive the user likely already owns. This lowers the adoption barrier relative to dedicated hardware tokens, at the cost of weaker tamper resistance than a purpose-built secure element: the key file on the USB drive is an ordinary, copyable file, and a party who obtains it can carry out an offline brute-force attack against the password-derived wrapping key, whereas a genuine hardware token never exposes raw key material for extraction under any circumstance.

2.1 Feature Comparison

Table 1 summarizes how Secure Vault’s feature set compares against representative tools from each of the three categories above. No single alternative combines two-factor (possession + knowledge) custody, zero additional hardware cost, at-rest encryption of the key token itself, keyed integrity verification, and full source auditability in one package; each existing category makes a different subset of these trade-offs.

The comparison illustrates Secure Vault’s specific niche: it is the only option in this set that is simultaneously two-factor, hardware-cost-free, file-level, fully open source, and portable across machines. The

trade-off, discussed further in Section 7, is that a commodity USB drive offers weaker physical tamper resistance than a purpose-built secure element inside a hardware token, since the key material is, in principle, extractable as a file rather than sealed inside tamper-resistant silicon.

Property	VeraCrypt (password-only)	BitLocker (TPM-bound)	Hardware Token (e.g. YubiKey)	Secure Vault
Two-factor (possession + knowledge)	No	Yes ^a	Yes	Yes
No dedicated hardware required	Yes	Yes	No	Yes
Survives loss of device alone	No	Yes	Yes	Yes
Key encrypted at rest on token	N/A	N/A	Yes	Yes
Keyed (HMAC) integrity verification	No	N/A	Varies	Yes
Purpose-separated key derivation (HKDF)	N/A	N/A	Yes	Yes
AAD-bound authentication (anti cross-context reuse)	No	N/A	Varies	Yes
Fully open source	Yes	No	No	Yes
Free of charge / no vendor lock-in	Yes	Yes ^b	No	Yes
File-level (as opposed to disk-level)	Yes	No	Varies	Yes
Portable across machines	Yes	No	Yes	Yes
Raw key material never extractable	No	N/A	Yes	No^c

Table 1: Feature comparison against representative tools in each encryption-approach category.

^a BitLocker’s default configuration binds to the TPM alone, without a second factor, unless PIN or startup-key mode is explicitly enabled by the administrator.

^b BitLocker is bundled with Windows Pro and Enterprise editions but is unavailable on Home editions without a paid upgrade.

^c Unlike a genuine hardware token, Secure Vault’s USB-resident key file is an ordinary, copyable file; encryption at rest raises the cost of misuse but does not achieve the non-extractability property of tamper-resistant hardware. See Section 4.2.

3 Threat Model

Secure Vault is designed to protect file confidentiality and integrity against the following adversary:

- **Remote attacker with no physical access.** An attacker who compromises the host machine remotely (malware, remote access, credential theft) but does not have physical possession of the registered USB drive cannot decrypt vault files, since the key material never leaves the drive in plaintext and is not derivable from anything stored on the host disk.
- **Opportunistic device thief.** An attacker who steals the laptop but not the USB drive (a common real-world scenario, since the two are rarely carried together) cannot decrypt vault files.
- **Attacker with the USB drive but not the password.** As of v3.3.0, the key on the USB drive is itself encrypted with a password-derived key via scrypt; possession of the drive alone is insufficient. This is removable-key possession rather than hardware-bound custody: the encrypted key file can be copied freely, and an attacker who obtains it can mount an unlimited, offline brute-force campaign against the password without any further access to the original drive.

Secure Vault explicitly does not defend against the following, and the project documentation states this openly:

- An attacker who obtains both the USB drive and the password (e.g., both written down together, or both stolen in the same theft).
- An attacker who copies the encrypted key file off the USB drive and subsequently mounts an offline brute-force or dictionary attack against the user’s password, unconstrained by rate-limiting, since `scrypt`’s memory-hardness raises cost per guess but does not prevent an attacker with sufficient resources from attempting many guesses in parallel.
- Forensic recovery of the plaintext original file from an SSD after “secure delete”, since wear-leveling on modern SSDs means an overwrite-based deletion routine cannot guarantee physical erasure at the hardware level.
- An attacker with kernel-level or physical access to the running host who can read process memory while the vault is unlocked, since some key material and, prior to v3.5.0, decrypted file content transiently existed in process memory during encryption/decryption operations despite explicit zeroing after use.

A deliberate consequence of this model is that there is no password recovery mechanism. Because the password is never stored anywhere, forgetting it renders any data encrypted under it permanently unrecoverable, even with the correct USB drive in hand. This is a usability cost that the project accepts explicitly in exchange for eliminating any server-side or local recovery secret that could itself become an attack target.

4 System Design

4.1 Cryptographic Primitives

Secure Vault is implemented in Node.js and relies exclusively on Node’s built-in `crypto` module rather than a third-party cryptographic library, minimizing the dependency-driven attack surface. The core primitives are:

- **AES-256-GCM** for authenticated encryption of file contents, providing confidentiality and built-in tamper detection via the GCM authentication tag.
- **`scrypt`** as the key-derivation function used to turn a user password into a key-encrypting key, chosen for its memory-hardness relative to PBKDF2, which raises the cost of GPU/ASIC-accelerated brute-force attacks.
- **HKDF (RFC 5869)**, introduced in v3.5.0, used to derive an encryption key and a distinct HMAC integrity key from the same unlocked master key, rather than reusing a single key for two cryptographically different purposes.
- **Keyed HMAC-SHA256** for file integrity verification, computed using the HKDF-derived integrity key and checked before any decryption is attempted.
- **Additional Authenticated Data (AAD) context binding**, introduced in v3.5.0, in which every AES-256-GCM operation binds a format/purpose identifier into its authentication tag, so that a ciphertext and tag produced for one context (e.g., the wrapped master-key layer on the USB drive) cannot be substituted into a different context (e.g., the file-content layer) without failing authentication.
- **Cryptographically random salts and initialization vectors**, generated fresh for every encryption operation, avoiding key or nonce reuse across files.

4.2 Key Custody Model

At registration, Secure Vault generates a random 32-byte master key and writes it, encrypted under a script-derived key from the user’s password, to a hidden file (`.vault_master.key`) at the root of a user-designated USB drive. The application supports registering a single master key alongside optional additional device-specific keys, allowing a user to maintain one master key while issuing separate keys per machine.

It is important to state precisely what this design does and does not achieve. The USB drive provides *removable-key possession*: the encrypted key file is an ordinary file that can be copied, backed up, or exfiltrated like any other, and the password is the sole barrier standing between a copy of that file and the underlying master key. This is a meaningfully different security property from a genuine hardware-bound token (e.g., a TPM or a FIDO2 device), in which raw key material is generated inside, and never leaves, tamper-resistant hardware — such devices can enforce rate-limiting, attempt counters, or physical destruction of key material on tampering, none of which is possible for a key stored as a file on commodity flash storage. Secure Vault’s design trades this stronger guarantee for zero hardware cost and full user portability; Section 7 and Section 8 discuss FIDO2/PIV-compatible hardware as a future, higher-assurance alternative for users who require non-extractable key custody.

Encryption and decryption operations both require the registered drive to be physically mounted; the software reads and decrypts the key into memory only for the duration of the operation. Passwords and key material are captured into a `Buffer` rather than a JavaScript string and explicitly zeroed (`.fill(0)`) immediately after use. As of v3.5.0, this zeroing discipline was extended to the decrypted plaintext file content buffer itself, not only key material, further reducing — though, as discussed in Section 3, not eliminating — the window during which sensitive data resides in process memory.

4.3 System Architecture

Figure 1 illustrates the end-to-end key-unlock and encryption flow. The user’s password and the USB-resident encrypted key file are two independent inputs that must both be supplied before the master key becomes available in memory; the master key then drives HKDF-based derivation of separate encryption and integrity keys, which in turn drive the AES-256-GCM encryption path (with AAD context binding) and the keyed HMAC-SHA256 integrity path, respectively.

Component	Responsibility
<code>secure-vault-launcher.js</code>	CLI entry point / interactive menu
<code>crypto.js</code>	AES-256-GCM encrypt/decrypt, scrypt KDF, HKDF, HMAC, AAD binding
<code>auth.js</code>	USB key registration, password unlock flow
<code>config/settings.json</code>	Email alert configuration (optional feature)
<code>config/registered-devices.json</code>	Registered USB key metadata
Windows Explorer shell extension	Optional right-click encrypt/decrypt

Table 2: Core system components and responsibilities.

Auxiliary features layered on top of the core cryptographic path include: automatic local encrypted backups with rotation (retaining the five most recent, expiring after 30 days) to guard against accidental data loss during normal use; a restore path independent of the decrypt operation; and an optional email notification feature (Gmail SMTP only, using an App Password rather than the account password) that alerts the user when a vault action occurs, useful for detecting unauthorized use of a lost or stolen drive if the host machine is still reachable.

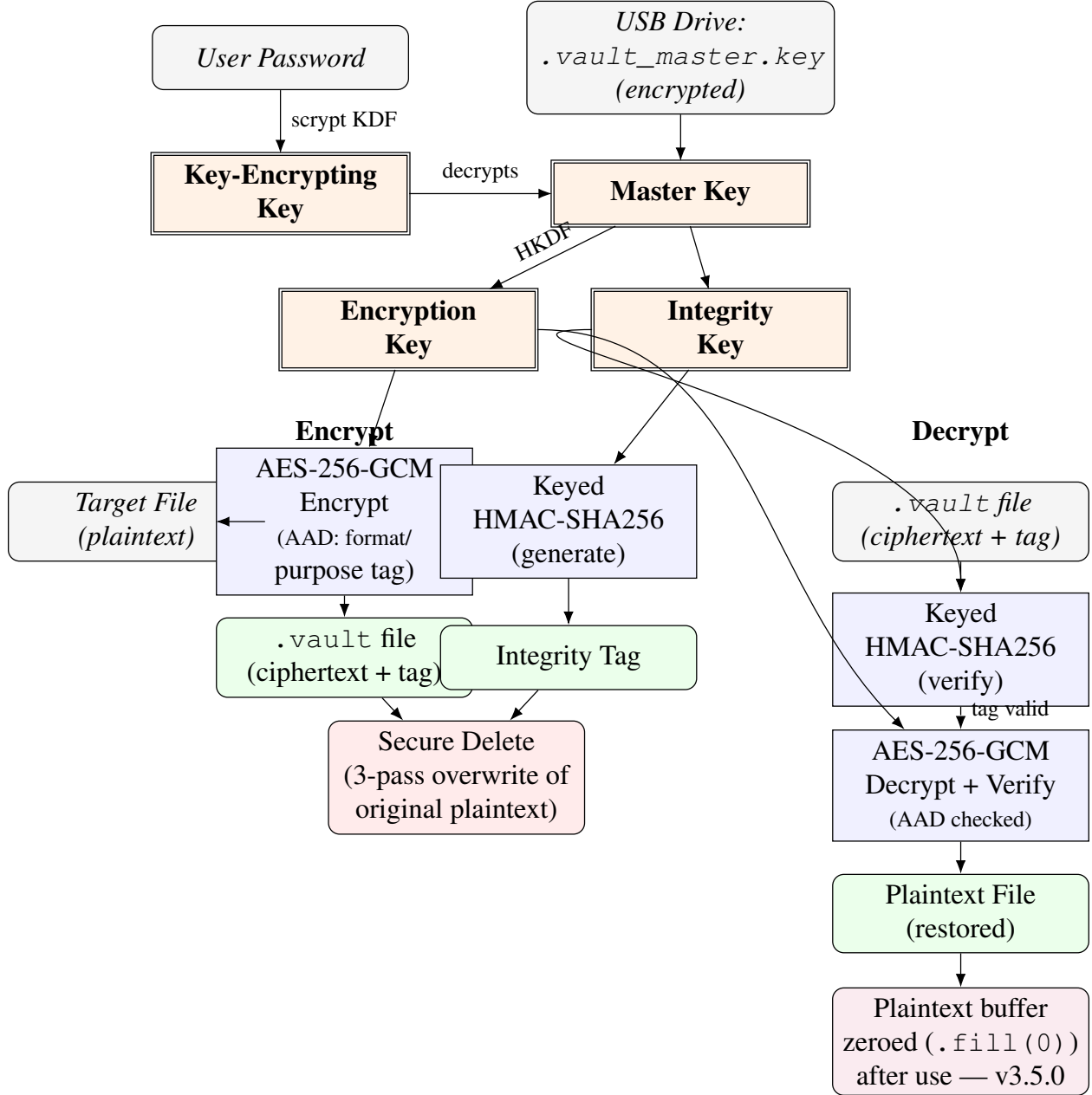


Figure 1: Secure Vault key-unlock, encryption, and decryption flow (v3.5.0). Double-border boxes denote secret key material held only transiently in process memory; rounded boxes denote inputs or final outputs; sharp-cornered shaded rectangles denote cryptographic operations. Password and USB-resident encrypted key are independent inputs (two-factor); only their combination unlocks the master key, from which HKDF derives independent encryption and integrity keys. *Left column (encrypt)*: the target file is encrypted under AES-256-GCM with a bound AAD context tag, a keyed HMAC-SHA256 tag is generated over the same key material, and the original plaintext is securely deleted only after both outputs are written. *Right column (decrypt)*: the stored integrity tag is verified first (dashed edge marks the gate — decryption proceeds only if the tag is valid), then AES-256-GCM decryption re-checks the AAD context before releasing plaintext; the resulting plaintext buffer is explicitly zeroed after use, a protection added in v3.5.0 that previously covered only key material.

4.4 Distribution and Build Pipeline

The project distributes a Windows installer built with Inno Setup, which silently installs the Node.js runtime if absent, installs the single production dependency (`nodemailer`), and registers Start Menu / optional shell-extension entries. The installer is built reproducibly via a GitHub Actions workflow triggered on version-tag push, which runs on a Windows runner and attaches the resulting executable directly to a GitHub Release, removing the need for a local Windows build environment and ensuring the published binary corresponds to a specific, auditable commit. As of this writing, the repository additionally runs GitHub CodeQL static analysis and Dependabot dependency scanning on every push to `main` (see Section 6).

5 Security Hardening: A Release History Case Study

Unlike a paper describing a system designed correctly from inception, Secure Vault’s public history documents a realistic vulnerability-and-remediation cycle typical of an actively maintained solo open-source security tool. We report this history because we believe the process is as informative as the end state.

Plaintext key-on-drive exposure (fixed v3.3.0). In earlier versions, the 32-byte master key written to the USB drive was stored as raw plaintext bytes. Any party with momentary physical access to the drive — for example, plugging it into another machine while unattended — could copy the key file directly, fully defeating the two-factor design regardless of password strength. As of v3.3.0, the key is encrypted at rest on the drive using AES-256-GCM with a script-derived key from the user’s password, so that drive possession alone is insufficient. The project’s README explicitly flags this as a breaking change: files encrypted under the old scheme require decrypting under the prior version before upgrading.

Forgeable integrity checksum (fixed prior to v3.4.0). File integrity was originally verified using an unkeyed SHA-256 checksum. Because SHA-256 alone provides no authentication, any party able to modify the ciphertext could recompute and replace the checksum to match, providing no real protection against tampering — only against accidental corruption. This was replaced with a keyed HMAC-SHA256 computed using the same unlocked key material, so that generating a valid checksum for tampered data requires possession of the key, closing the gap between “integrity check” and “integrity check that resists an adversary.”

Shell command injection via 7-Zip invocation (fixed prior to v3.4.0). Earlier versions invoked the bundled 7-Zip binary by constructing a shell command string that embedded the target file or folder name. A maliciously crafted filename could break out of shell quoting and inject arbitrary commands. The fix switched to `execFileSync` with arguments passed as an array rather than a concatenated string, which bypasses shell interpretation entirely and eliminates this class of injection.

Insufficient key-derivation cost (fixed v3.4.0). Following an internal focused review of `crypto.js` and `auth.js`, the script cost parameter was doubled ($2^{15} \rightarrow 2^{16}$), raising the computational cost of an offline brute-force attack against the password-derived key-encrypting key, at a modest increase in legitimate unlock latency. This release also added explicit input-length validation on the decrypt path and fixed a multi-drive key-detection bug in which a drive letter typed without a trailing colon (e.g., `e` instead of `E:`) was misinterpreted as a relative folder path rather than a drive reference.

Cross-purpose key reuse and missing context binding (fixed v3.5.0). A subsequent design review identified that the master key was being used directly for both AES-256-GCM encryption and HMAC-SHA256 integrity verification, a cryptographic anti-pattern in which a single key serves two distinct purposes; while no concrete exploit was demonstrated against this specific configuration, reusing one key across independent primitives is widely discouraged because it removes the cryptographic separation that would otherwise contain a weakness discovered in one primitive to that primitive alone. Separately, AES-256-GCM ciphertexts and tags were not bound to the context in which they were generated, meaning a ciphertext and authentication tag produced for one purpose (e.g., wrapping the master key on the USB drive) could,

in principle, be substituted into a different decryption call site (e.g., file-content decryption) without failing authentication, provided the underlying key matched. v3.5.0 addresses both issues: encryption and integrity keys are now derived independently via HKDF from the same root key, and every AES-256-GCM operation binds a format/purpose identifier as Additional Authenticated Data, so that ciphertext/tag pairs from one context are cryptographically rejected if presented in another. This release also extended the existing buffer-zeroing discipline to decrypted plaintext file content, not only key material.

5.1 Discussion

The five fixes documented above span four categorically different vulnerability and design-hardening classes — key custody, integrity authentication, injection, and cryptographic key/context separation — which suggests they were not caught by a single review methodology or tool, but through cumulative scrutiny across releases. This is consistent with the broader observation that small, security-focused open-source tools benefit disproportionately from public code review even absent a formal audit budget: several of these issues are the kind that automated static analysis (e.g., CodeQL’s injection-pattern detectors) can catch, while others — the plaintext-key design flaw, and the key-reuse/AAD-binding issues addressed in v3.5.0 — are architectural and require human cryptographic reasoning rather than pattern matching. The v3.5.0 changes in particular illustrate that hardening does not stop once obvious implementation bugs are fixed; even a system built on correct primitives (AES-256-GCM, `scrypt`, HMAC) can carry design-level weaknesses in how those primitives are composed, and closing those gaps required a dedicated review pass focused specifically on key and context separation rather than on individual function correctness.

6 Continuous Assurance in Lieu of Formal Audit

Secure Vault has not undergone an independent third-party security audit, and the project documentation states this without qualification. In its place, the repository runs GitHub CodeQL static analysis (covering both JavaScript/TypeScript source and the Actions workflow definitions themselves) on every push, with zero outstanding alerts at the time of writing, and Dependabot dependency scanning, which most recently drove a version pin and patch of the project’s sole runtime dependency, `nodemailer`, closing eight flagged advisories spanning SMTP injection, CRLF injection, SSRF, denial-of-service, and TLS-verification-bypass classes.

We are explicit that this is not equivalent to a manual audit: static analysis catches syntactic and pattern-based vulnerability classes reliably, but is not a substitute for a human reviewer reasoning about protocol-level design flaws, as was the case with the original plaintext key-storage decision and the key-reuse/AAD-binding issues addressed in v3.5.0 (Section 5). We present continuous automated scanning as a meaningfully raised floor for a project at this budget level, not as a certification, and we recommend that any user evaluating Secure Vault for sensitive use weigh this limitation explicitly.

6.1 Current Repository Status

As of this writing, the following automated checks are live on the `main` branch and run on every push:

- **CodeQL (JavaScript/TypeScript analysis)** — static analysis of `crypto.js`, `auth.js`, `secure-vault-launcher.js`, and all other application source. Currently passing with zero open alerts.
- **CodeQL (GitHub Actions analysis)** — static analysis of the repository’s own CI/CD workflow definitions, guarding against script-injection and misconfiguration in the build pipeline itself. Currently passing with zero open alerts.

- **Automated installer build** — the Inno Setup build workflow runs on every push and version tag, confirming the installer still compiles successfully from the current source tree before it is ever attached to a release.
- **Dependabot dependency monitoring** — continuously tracks the project’s dependency tree for newly disclosed CVEs and opens automated patch pull requests; the `nodemailer` patch described above was delivered through this mechanism.

These checks were not present at the time of the project’s earliest releases; their introduction is itself part of the hardening trajectory documented in Section 5, and their continued green status is tracked publicly via the repository’s Actions history rather than asserted in this paper alone.

7 Limitations

We summarize the system’s known limitations, several of which are architectural rather than implementation bugs and are unlikely to be fully resolved without a materially different design:

1. **Removable-key custody, not hardware-bound custody.** As discussed in Section 4.2, the USB-resident key file is an ordinary, copyable file. Encryption at rest (v3.3.0) raises the cost of misuse by requiring the password as well, but does not achieve the non-extractability guarantee of a genuine hardware security token, which never permits raw key material to leave the device.
2. **No password recovery.** By design; see Section 3.
3. **SSD secure-delete limitations.** Multi-pass overwrite of the original plaintext file before deletion does not guarantee unrecoverability on SSDs due to wear-leveling, which may retain copies of overwritten data in blocks not currently mapped to the logical file.
4. **Residual memory exposure.** Explicit buffer zeroing — extended in v3.5.0 to plaintext file content as well as key material — reduces, but does not eliminate, the risk of sensitive data persisting in process memory, due to factors outside application control such as V8 garbage collection behavior and OS-level page swapping.
5. **Unsigned installer.** The distributed Windows installer is not code-signed, which triggers a Windows SmartScreen warning on first run. This is a distribution-trust issue rather than a cryptographic weakness, but it materially affects the tool’s perceived legitimacy to non-technical users and is one of the primary remaining items before wider public distribution.
6. **Single-platform scope.** The system currently targets Windows only, via an Inno Setup installer and Windows Explorer shell integration.
7. **No formal, independent audit.** See Section 6.

8 Conclusion and Future Work

Secure Vault demonstrates that removable-key custody using commodity USB storage, combined with standard authenticated-encryption primitives available in a language runtime’s built-in cryptography library, is a practical and low-cost way to raise the security bar for individual file encryption without requiring dedicated cryptographic hardware. The project’s six-release hardening history illustrates both the value and the limits of iterative, community-visible security review for a solo-maintained tool: architectural flaws (plaintext key

storage, cross-purpose key reuse, missing context binding) and implementation flaws (forgeable checksums, injection vectors) were each identified and fixed, but only after initial release, underscoring that continuous scrutiny — not a single design pass — is what closes the gap between “intended to be secure” and “demonstrated to be reasonably secure.”

Future work includes: pursuing code signing to remove the SmartScreen distribution barrier; seeking an independent, ideally funded, third-party security audit of the `crypto.js` and `auth.js` modules specifically; extending platform support beyond Windows; and evaluating FIDO2/PIV-compatible hardware tokens as an optional higher-assurance alternative to commodity USB storage for users who require genuine non-extractable key custody rather than the removable-key possession model described in Section 4.2.

Availability

Source code, release history, and full documentation are publicly available at:

<https://github.com/keerthivasan-sankar/secure-vault-for-commercial>, under the MIT license.

References

- [1] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.
- [2] C. Percival, *Stronger Key Derivation via Sequential Memory-Hard Functions*, BSDCan 2009.
- [3] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS PUB 197, 2001.
- [4] H. Krawczyk, M. Bellare, R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, IETF, 1997.
- [5] H. Krawczyk, P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869, IETF, 2010.
- [6] IDRIX, *VeraCrypt Documentation*, <https://www.veracrypt.fr/en/Documentation.html>.
- [7] GitHub, Inc., *CodeQL Documentation*, <https://codeql.github.com/docs/>.